

原

时间序列模型Prophet使用详细讲解

2018年10月26日 20:39:34 anshuai_aw1 阅读数 9517

 版权声明：https://blog.csdn.net/anshuai_aw1/article/details/83412058

之前我们已经讲过了如何在Windows系统下安装Python版本的Prophet。详细见[这里](#)。

接下来的几个部分，我们说下如何使用Prophet，以此来体验下Prophet的丰富内容。内容会比较多，主要翻译自[官方文档](#)。文档中使用的数据集可在[github 主页](#) 中的 [examples 文件夹](#) 内下载得到。

目录

一、简易入门

二、饱和预测

2.1 预测饱和增长

2.2 预测饱和减少

三、趋势突变点

3.1 Prophet 中的自动监测突变点

3.2 调整趋势的灵活性

3.3 指定突变点的位置

四、季节性，假期效果和回归量

4.1 对假期和特征事件建模

4.2 季节性的傅里叶级数

4.3 自定义季节性因素

4.4 对节假日和季节性设定先验规模

4.5 附加的回归量

五、乘法季节性

六、预测区间

6.1 趋势的不确定性

6.2 季节的不确定性

七、异常值

八、非日数据

8.1 子日数据

8.2 有规则间隔的数据

8.3 月数据

九、诊断

十、与机器学习算法的对比

👍 7

💬 6

📖

🔖

📱

...

一、简易入门

Prophet 遵循 **sklearn** 库建模的应用程序接口。我们创建了一个 Prophet 类的实例，其中使用了“拟合模型” **fit** 和“预测” **predict** 方法。

Prophet 的输入量往往是一个包含两列的数据框：**ds** 和 **y**。**ds** 列必须包含日期（YYYY-MM-DD）或者是具体的时间点（YYYY-MM-DD HH:MM:S），**y** 列必须是数值变量，表示我们希望去预测的量。

下面实例中使用的是 [佩顿·曼宁的维基百科主页](#) 每日访问量的时间序列数据（2007/12/10 - 2016/01/20）。我们使用 [fbprophet](#) 的 `Wikipediatrend` 包加载这个数据集。这个数据集具有多季节周期性、不断变化的增长率和可以拟合特定日期（例如佩顿·曼宁的决赛和超级碗）的情况等不错的例子。（注：佩顿·曼宁为前美式橄榄球四分卫）

首先，我们导入数据，该数据已经做过了log处理（即，做过`df['y'] = np.log(df['y'])`操作）。

```
1 # Python
2 import pandas as pd
3 from fbprophet import Prophet
4 # 读入数据集
5 df = pd.read_csv('examples/example_wp_log_peyton_manning.csv')
6 df.head()
```



通过对一个 Prophet 对象进行实例化来拟合模型，任何影响预测过程的设置都将在构造模型时被指定。接下来，就可以使用 `fit` 方法代入历史数据集，拟合过程应当花费 1 - 5 秒。

```
1 # 拟合模型
2 m = Prophet()
3 m.fit(df)
```

预测过程则需要建立在包含日期 **ds** 列的数据框基础上。通过使用辅助的方法 `Prophet.make_future_dataframe` 来将未来的日期扩展指定的天数的数据框。默认情况下，这样做会自动包含历史数据的日期，因此我们也可以用来查看模型对于历史数据的拟合效果。

```
1 # 构建待预测日期数据框，periods = 365 代表除历史数据的日期外再往后推 365 天
2 future = m.make_future_dataframe(periods=365)
3 future.tail()
```

`predict` 方法将会对每一行未来 **future** 日期得到一个预测值（称为 **yhat**）。如果你传入了历史数据的日期，它将会提供样本的模型拟合值。预测 **forecast** 创建的对象应当是一个新的数据框，其中包含一列预测值 **yhat**，以及成分的分析 and 置信区间。

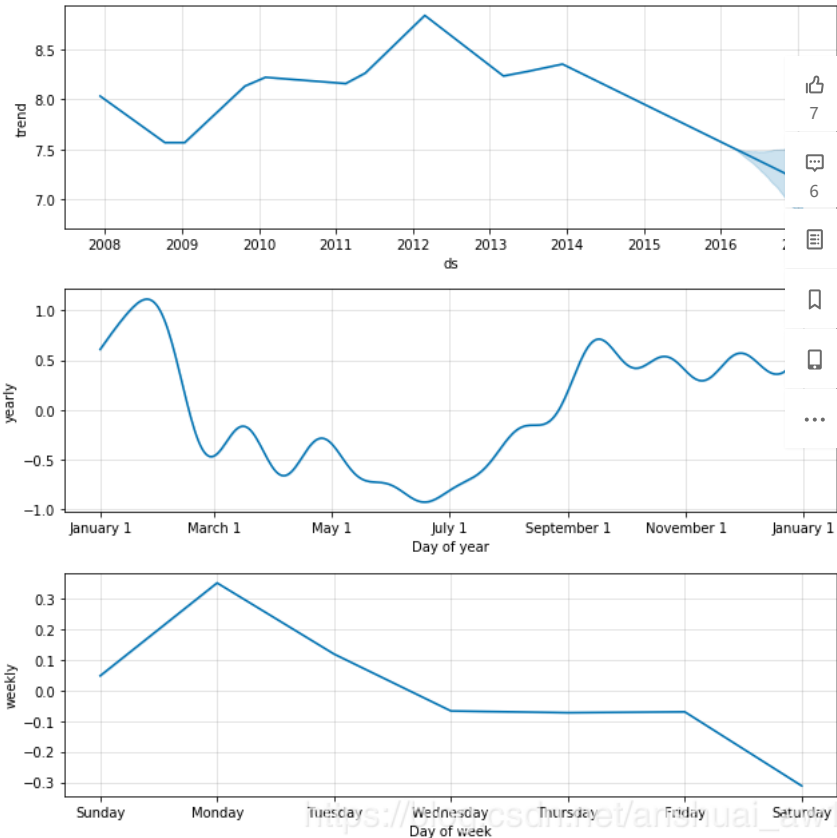
```
1 # 预测数据集
2 forecast = m.predict(future)
3 forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']].tail()
```

通过 `Prophet.plot` 方法传入预测得到的数据框，可以对预测的效果进行绘图。

```
1 # 展示预测结果
2 m.plot(forecast);
```

如果想查看预测的成分分析，可以使用 `Prophet.plot_components` 方法。默认情况下，将展示趋势、时间序列的年度季节性和周季节性。如果有的话，也会展示出来。

```
1 # 预测的成分分析绘图，展示预测中的趋势、周效应和年度效应
2 m.plot_components(forecast);
```



注：一个很核心的问题是我们应该怎么样理解上图的3个子图。通过对forecast这个Dataframe分析我们就可以得到结论。

我们先看下forecast都有哪些列。

```
1 print(forecast.columns)
2 Index(['ds', 'trend', 'trend_lower', 'trend_upper', 'yhat_lower', 'yhat_upper',
3       'additive_terms', 'additive_terms_lower', 'additive_terms_upper',
4       'multiplicative_terms', 'multiplicative_terms_lower',
5       'multiplicative_terms_upper', 'weekly', 'weekly_lower', 'weekly_upper',
6       'yearly', 'yearly_lower', 'yearly_upper', 'yhat'],
7       dtype='object')
```

通过对数据的分析，我们就可以知道：

①图1是根据trend画出来的，图2是根据weekly画出来的，图3是根据yearly画出来的。

②因为是加法模型，有： $\text{forecast}[\text{'additive_terms'}] = \text{forecast}[\text{'weekly'}] + \text{forecast}[\text{'yearly'}]$ ；有： $\text{forecast}[\text{'yhat'}] = \text{forecast}[\text{'trend'}] + \text{forecast}[\text{'additive_terms'}]$ 。因此： **$\text{forecast}[\text{'yhat'}] = \text{forecast}[\text{'trend'}] + \text{forecast}[\text{'weekly'}] + \text{forecast}[\text{'yearly'}]$** 。

如果有节假日因素，那么就会有 $\text{forecast}[\text{'yhat'}] = \text{forecast}[\text{'trend'}] + \text{forecast}[\text{'weekly'}] + \text{forecast}[\text{'yearly'}] + \text{forecast}[\text{'holidays'}]$ 。

在第四部分，我们会讲到节假日因素，对于那些是节假日的天数， $\text{forecast}[\text{'holidays'}]$ 才会有值，不是节假日的天数， $\text{forecast}[\text{'holidays'}]$ 为0。

③因为是加法模型，'multiplicative_terms', 'multiplicative_terms_lower', 'multiplicative_terms_upper'这3列为空。

因此，基于上面的分析，weekly中的Monday为0.3的意思就是，在trend的基础上，加0.3；Saturday为-0.3的意思就是，在trend的基础上，减0.3。的高低也在一定程度上反应了“销量的趋势”。

注：许多方法的细节可以通过`help(Prophet)` 或者 `help(Prophet.fit)` 来获得。

二、饱和预测

2.1 预测饱和增长

默认情况下，Prophet 使用线性模型进行预测。当预测增长情况时，通常会存在可到达的最大极限值，例如：总市场规模、总人口数等等。这被称做（carrying capacity），那么预测时就应当在接近该值时趋于饱和。

Prophet 可使用 **logistic 增长** 趋势模型进行预测，同时指定承载能力。下面使用 **R 语言的维基百科主页** 访问量（取对数）的实例来进行说明。

首先，我们导入数据，该数据已经做过了log处理（即，做过 $\text{df['y']} = \text{np.log(df['y'])}$ 操作）。

```
1 import pandas as pd
2 from fbprophet import Prophet
3 df = pd.read_csv('examples/example_wp_log_R.csv')
```

新建一列 **cap** 来指定承载能力的大小。本实例中假设取某个特定的值，**通常情况下这个值应当通过市场规模的数据或专业人士来决定。**

```
df['cap'] = 8.5
```

值得注意的是数据框的每行都必须指定 cap 的值，但并非需要是恒定值。如果市场规模在不断地增长，那么 **cap** 也可以是一个增长的序列。

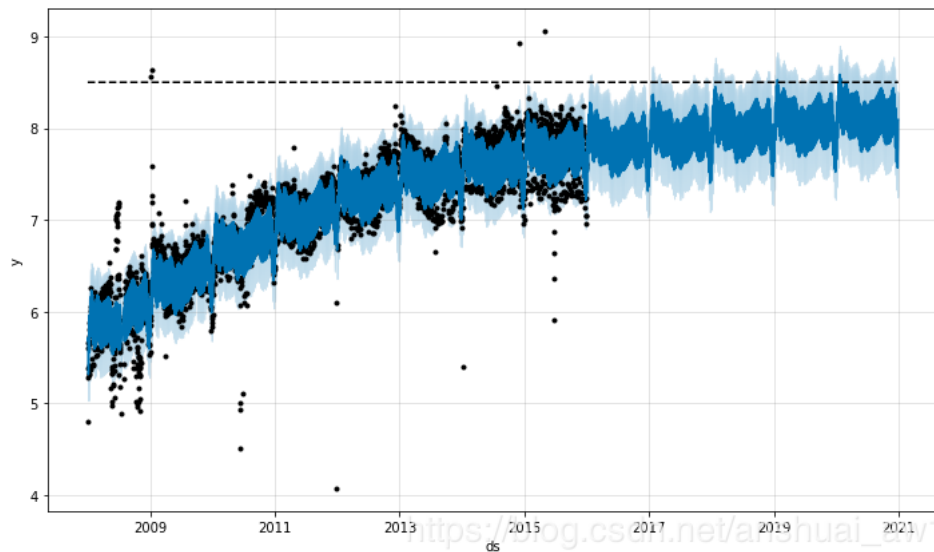
如之前教程所述来拟合模型，并且通过一个新增的参数来指定采用 logistic 增长：

```
1 m = Prophet(growth='logistic')
2 m.fit(df)
```

如前所述，我们可以创建一个数据框来预测未来值，不过这里要指定未来的承载能力。我们将未来的承载能力设定得和历史数据一样，并且预测未来 3

```
1 future = m.make_future_dataframe(periods=1826)
2 future['cap'] = 8.5
3 fcst = m.predict(future)
4 fig = m.plot(fcst)
```

预测结果如下图所示：

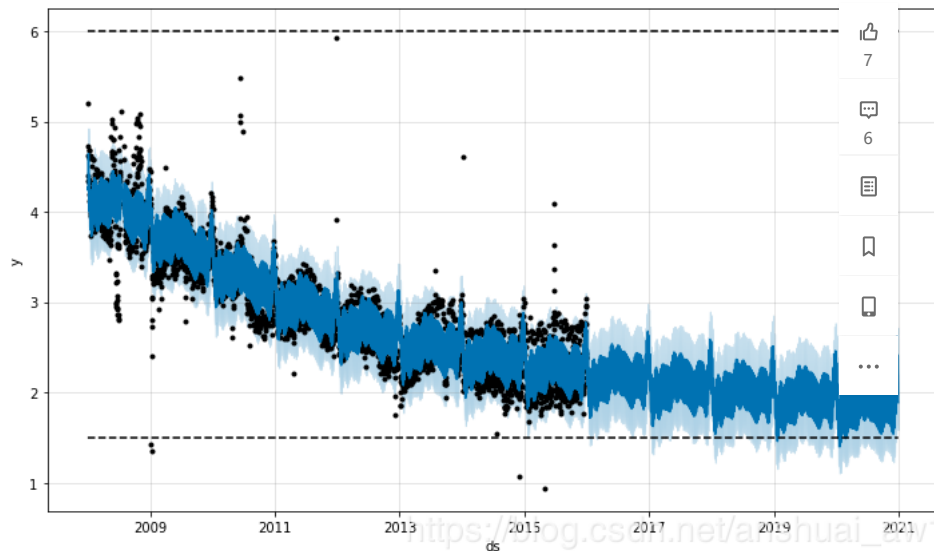


2.2 预测饱和和减少

logistic增长模型还可以处理饱和和最小值，方法与指定最大值的列的方式相同：

```
1 # Python
2 df['y'] = 10 - df['y']
3 df['cap'] = 6
4 df['floor'] = 1.5
5 future['cap'] = 6
6 future['floor'] = 1.5
7 m = Prophet(growth='logistic')
8 m.fit(df)
9 fcst = m.predict(future)
10 fig = m.plot(fcst)
```

结果如下图：



三、趋势突变点

在之前的部分，我们可以发现真实的时间序列数据往往在趋势中存在一些突变点。默认情况下，Prophet 将自动监测到这些点，并对趋势做适当地调整。但是，如果对趋势建模时发生了一些问题，例如：Prophet 不小心忽略了一个趋势速率的变化或者对历史数据趋势变化存在过拟合现象。如果我们希望对趋势更好地控制的话，那么下面将会介绍几种可以使用的方法。

3.1 Prophet 中的自动监测突变点

Prophet 首先是通过在大量潜在的突变点（变化速率突变）中进行识别来监测突变点的。之后对趋势变化的幅度做稀疏先验（等同于 L1 正则化）——Prophet 在建模时会存在很多变化速率突变的点，但只会尽可能少地使用它们。以第一部分中佩顿·曼宁的数据为例，默认下，Prophet 会识别出 21 个突变点（均匀分布在在前 80% 的时间序列数据中）。下图中的竖线指出这些潜在的突变点所在的位置。

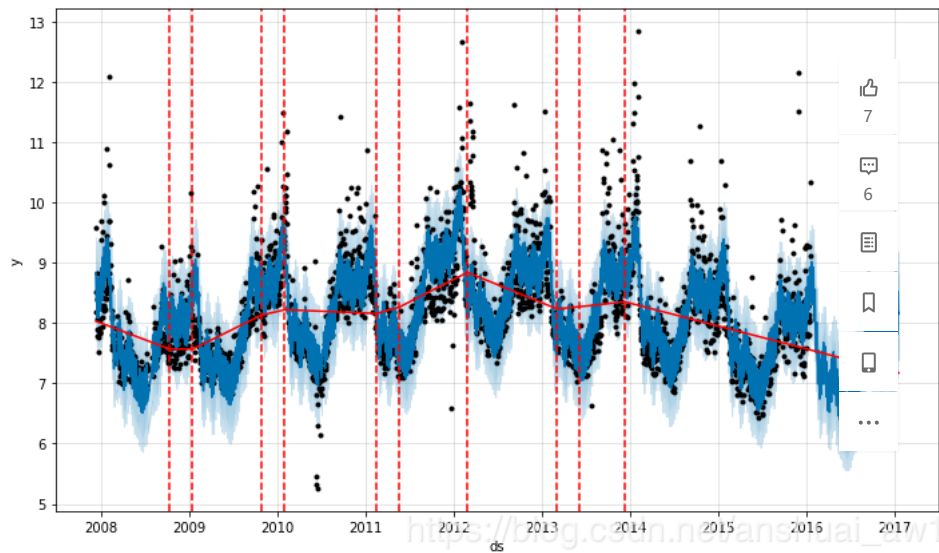
虽然存在很多变化速率可能会突变的点，但由于做了稀疏先验，绝大多数突变点并不会包含在建模过程中。如下图所示，通过观察对每个突变点绘制的图，可以发现这一点。

潜在突变点的数量可以通过设置 `n_changepoints` 参数来指定，但最好还是利用调整正则化过程来修正。

显著的突变点的位置可以通过以下代码获得：

```
1 # Python
2 from fbprophet.plot import add_changepoints_to_plot
3 fig = m.plot(forecast)
4 a = add_changepoints_to_plot(fig.gca(), m, forecast)
```





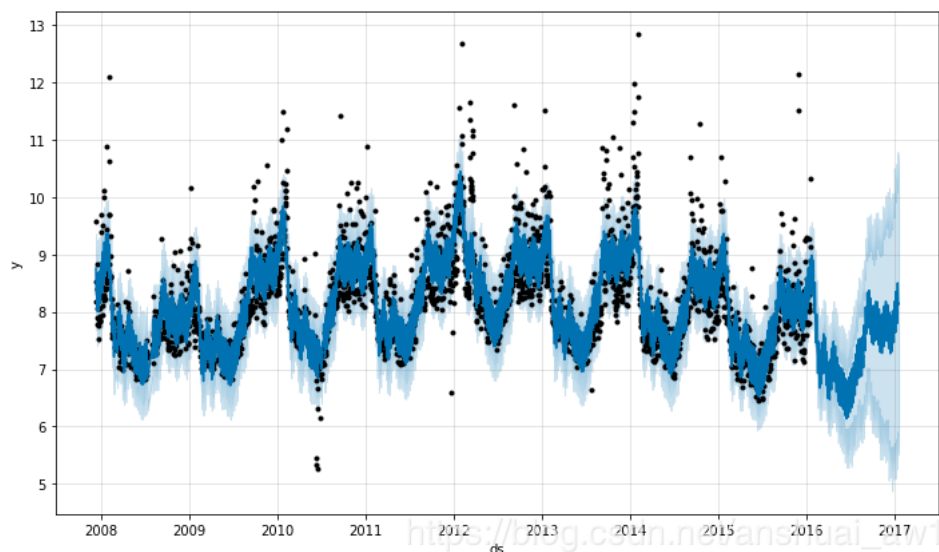
默认情况下，只有在时间序列的前80%才会推断出突变点，以便有足够的长度来预测未来的趋势，并避免在时间序列的末尾出现过度拟合的波动。这个在很多情况下工作，但不是所有情况下都可以，可以使用`changepoint_range`参数进行更改。例如，Python中的`m = Prophet(changepoint_range` 意味着将在时间序列的前90%处寻找潜在的变化点。

3.2 调整趋势的灵活性

如果趋势的变化被过度拟合（即过于灵活）或者拟合不足（即灵活性不够），可以利用输入参数 `changepoint_prior_scale` 来调整稀疏先验的程度。这个参数被指定为 0.05。

增加这个值，会导致趋势拟合得更加灵活。代码和图如下所示：

```
1 df = pd.read_csv('examples/example_wp_log_peyton_manning.csv')
2
3 # 拟合模型
4 m = Prophet(changepoint_prior_scale=0.5)
5 m.fit(df)
6
7 future = m.make_future_dataframe(periods=365)
8
9 forecast = m.predict(future)
10 fig = m.plot(forecast)
```



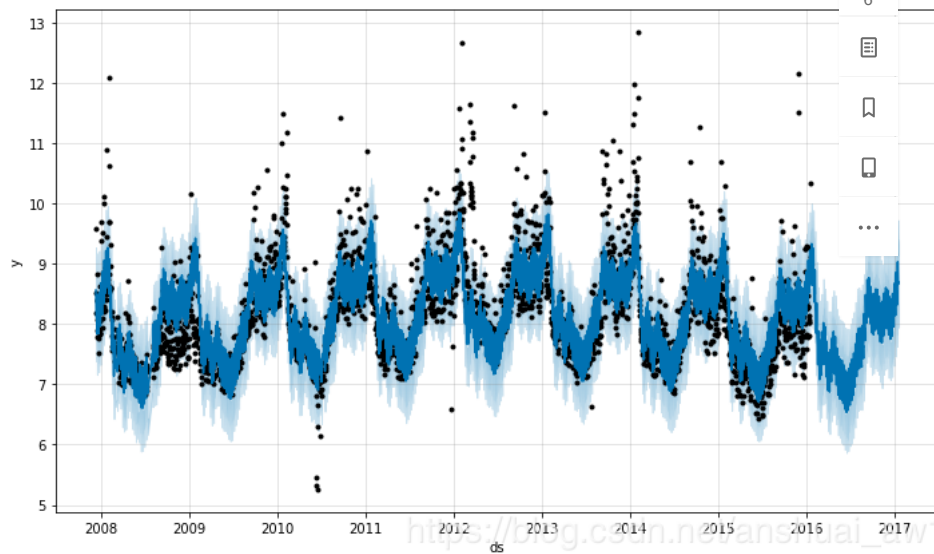
减少这个值，会导致趋势拟合得灵活性降低。代码和图如下所示：

```
1 m = Prophet(changepoint_prior_scale=0.001)
2 m.fit(df)
```

```

3 | future = m.make_future_dataframe(periods=365)
4 | forecast = m.predict(future)
5 | m.plot(forecast);

```



3.3 指定突变点的位置

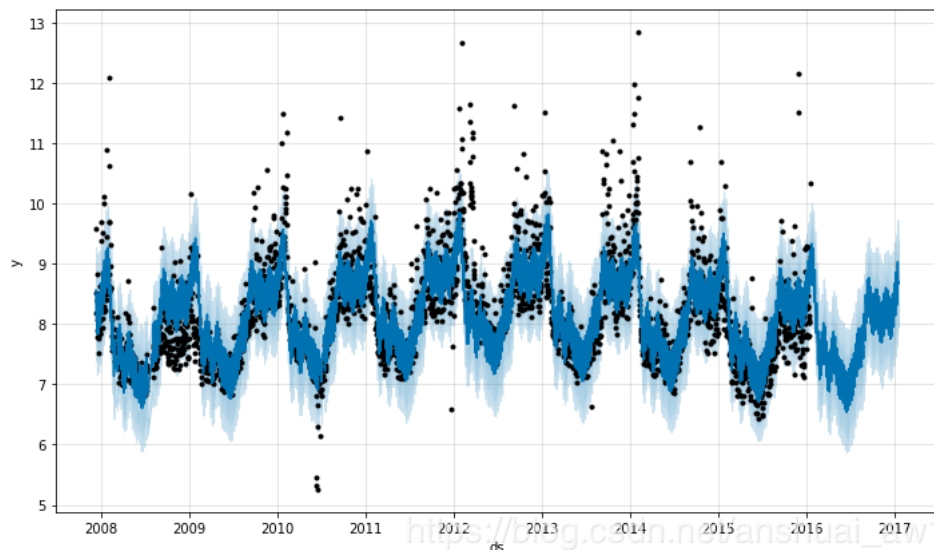
如果你希望手动指定潜在突变点的位置而不是利用自动的突变点监测，可以使用 `changepoints` 参数。

代码和图如下所示：

```

1 | m = Prophet(changepoints=['2014-01-01'])
2 | m.fit(df)
3 | future = m.make_future_dataframe(periods=365)
4 | forecast = m.predict(future)
5 | m.plot(forecast);

```



四、季节性，假期效果和回归量

4.1 对假期和特征事件建模

如果需要专门对节假日或者其它的事件进行建模，你就必须得为此创建一个新的dataframe，其中包含两列（节假日 `holiday` 和日期戳 `ds`），每行一个出现的节假日。**这个数据框必须包含所有出现的节假日，不仅是历史数据集中还是待预测的时期中的。**如果这些节假日并没有在待预测的时期中被注 Prophet 也会利用历史数据对它们建模，但预测未来时却不会使用这些模型来预测。

注：也就是说，在待预测的日期里，我们也必须指定所有出现的节假日。

你可以在这个数据框基础上再新建两列 `lower_window` 和 `upper_window`，从而将节假日的时间扩展成一个区间 [`lower_window`, `upper_window`]。如果说，如果想将平安夜也加入到“圣诞节”里，就设置 `lower_window = -1`，`upper_window = 0`；如果想将黑色星期五加入到“感恩节”里，设置 `lower_window = 0`，`upper_window = 1`。

下面我们创建一个数据框，其中包含了所有佩顿·曼宁参加过的决赛日期：

```
1 | playoffs = pd.DataFrame({
2 |     'holiday': 'playoff',
3 |     'ds': pd.to_datetime(['2008-01-13', '2009-01-03', '2010-01-16',
4 |                             '2010-01-24', '2010-02-07', '2011-01-08',
5 |                             '2013-01-12', '2014-01-12', '2014-01-19',
6 |                             '2014-02-02', '2015-01-11', '2016-01-17',
7 |                             '2016-01-24', '2016-02-07']),
8 |     'lower_window': 0,
9 |     'upper_window': 1,
10 | })
11 | superbowls = pd.DataFrame({
12 |     'holiday': 'superbowl',
13 |     'ds': pd.to_datetime(['2010-02-07', '2014-02-02', '2016-02-07']),
14 |     'lower_window': 0,
15 |     'upper_window': 1,
16 | })
17 | holidays = pd.concat((playoffs, superbowls))
```

上述代码中，我们将超级碗的日期既记录在了决赛的日期数据框中，也记录在了超级碗的日期数据框中。这就会造成超级碗日期的效应会在决赛日期附近发生。

一旦这个数据框创建好了，就可以通过传入 `holidays` 参数使得在预测时考虑上节假日效应。这里我们仍以第一部分中佩顿·曼宁的数据为例：

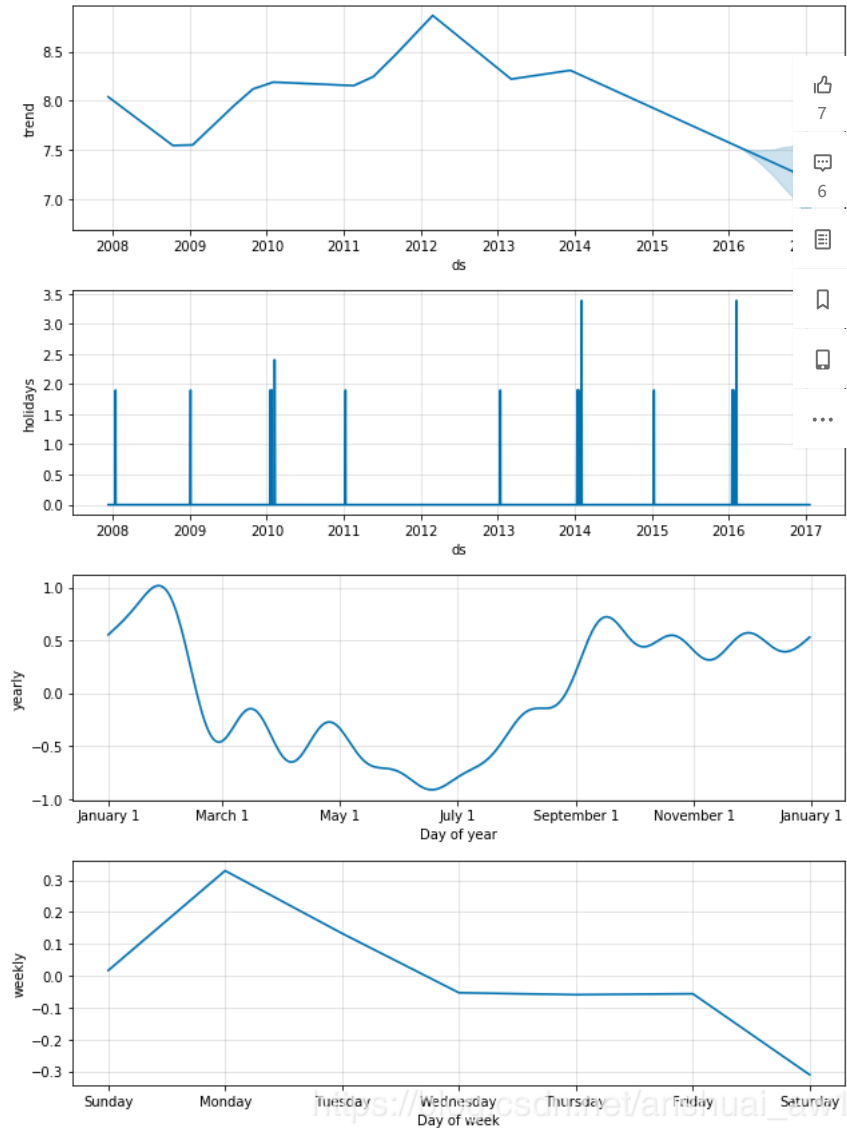
```
1 | df = pd.read_csv('examples/example_wp_log_peyton_manning.csv')
2 |
3 | m = Prophet(holidays=holidays)
4 | m.fit(df)
5 | future = m.make_future_dataframe(periods=365)
6 | forecast = m.predict(future)
```

可通过 `forecast` 数据框，来展示节假日效应：

```
1 | # 看一下假期的最后10行数据
2 | forecast[(forecast['playoff'] + forecast['superbowl']).abs() > 0][
3 |     ['ds', 'playoff', 'superbowl'][-10:]]
```

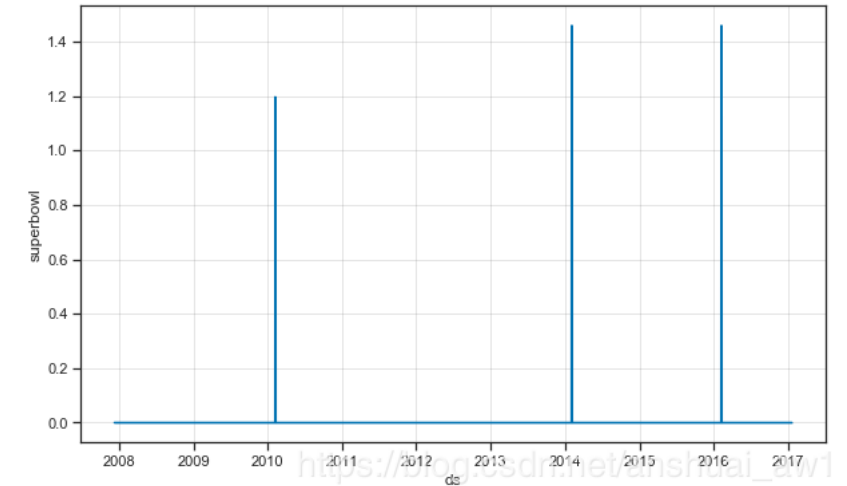
在成分分析的图中，如下所示，也可以看到节假日效应。我们可以发现，在决赛日期附近有一个穿透，而在超级碗日期时穿透则更为明显。

```
fig = m.plot_components(forecast)
```

可以使用 `plot_forecast_component` (从 `fbprophet.plot` 导入) 来画出独立的节假日的成分。类似如下代码：

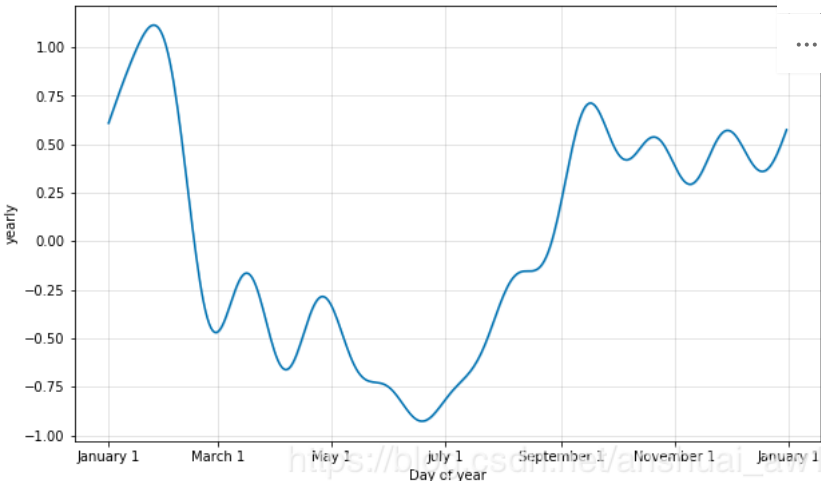
```
1 | from fbprophet.plot import plot_forecast_component
2 | m.plot_forecast_component(forecast, 'superbowl')
```



4.2 季节性的傅里叶级数

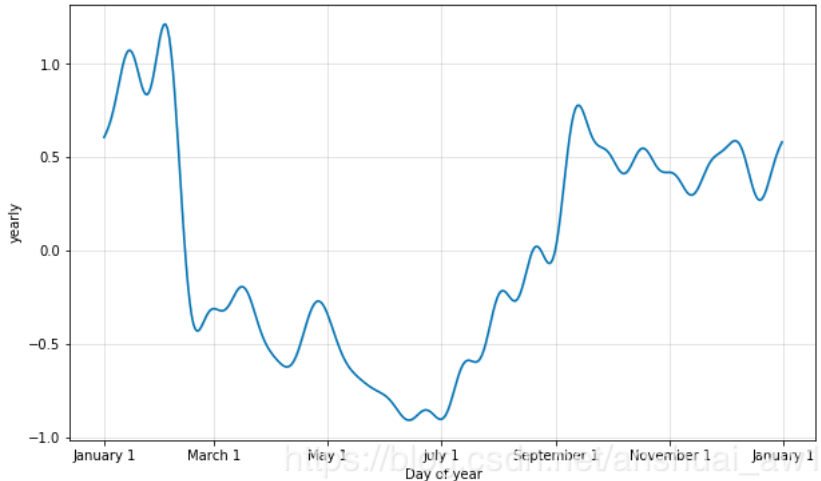
季节性是用部分傅里叶和估计的。有关完整的细节，请参阅论文，以及维基百科上的这个图，以说明部分傅里叶和如何近似于一个线性周期信号。部分的项数是一个参数，它决定了季节性的变化有多快。为了说明这一点，我们仍似乎用第一部分中佩顿·曼宁的数据。每年季节性的默认傅立叶级数是1了这样的拟合：

```
1 # Python
2 from fbprophet.plot import plot_yearly
3 m = Prophet().fit(df)
4 a = plot_yearly(m)
```



默认值10通常是合适的，但是当季节性需要适应更高频率的变化时，它们可以增加，并且通常不那么平滑。在实例化模型时，可以为每个内置季节性指数，这里增加到20：

```
1 from fbprophet.plot import plot_yearly
2 m = Prophet(yearly_seasonality=20).fit(df)
3 a = plot_yearly(m)
```



可以看到，曲线更加的多变了。增加傅立叶项的数量可以使季节性适应更快的变化周期，但也可能导致过度拟合：N个傅立叶项对应于用于建模的2

4.3 自定义季节性因素

如果时间序列超过两个周期，Prophet将默认适合每周和每年的季节性。对于子日（sub-daily）时间序列，它也将适合每日的季节性。在Python中，使用add_seasonality方法添加其它季节性(如每月、每季、每小时)。

这个函数的输入是一个名字，季节性的周期，以及季节性的傅里叶order。作为参考，默认情况下，Prophet为周季节性设定的傅立叶order为3，为年为10。add_seasonality的一个可选输入是该季节性组件的先验规模。

作为一个例子，我们仍使用佩顿·曼宁的数据，但是用每月的季节性替换每周的季节性。每月的季节性将出现在组件图中：

```

1 m = Prophet(weekly_seasonality=False)
2 m.add_seasonality(name='monthly', period=30.5, fourier_order=5)
3 forecast = m.fit(df).predict(future)
4 fig = m.plot_components(forecast)

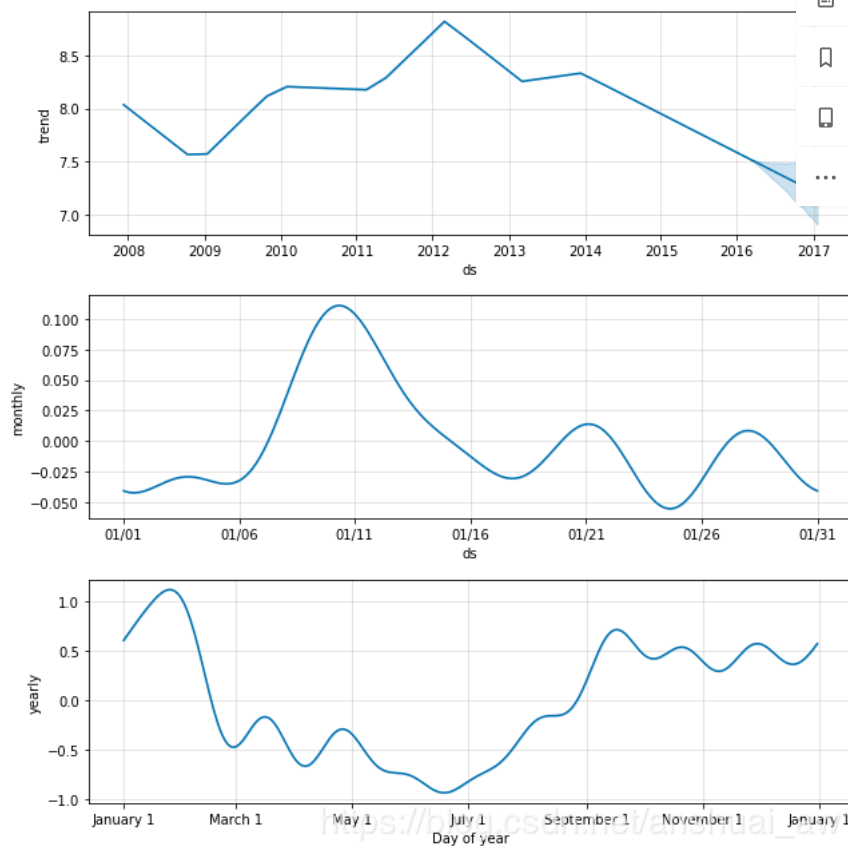
```



7



6



4.4 对节假日和季节性设定先验规模

如果发现节假日效应被过度拟合了，通过设置参数 `holidays_prior_scale` 可以调整它们的先验规模来使之平滑，默认下该值取 10。

减少这个参数会降低假期效果:

```

1 m = Prophet(holidays=holidays, holidays_prior_scale=0.05).fit(df)
2 forecast = m.predict(future)
3 forecast[(forecast['playoff'] + forecast['superbowl']).abs() > 0][
4     ['ds', 'playoff', 'superbowl']][-10:]

```

代码输出结果如下所示：

和之前相比，节假日效应的规模被减弱了，特别是对观测值较少的超级碗而言。类似的，还有一个 `seasonality_prior_scale` 参数可以用来调整季节性的拟合程度。

可以通过在节假日的dataframe中包含一个列 `prior_scale` 来单独设置先验规模。独立的季节性的先验规模可以作为 `add_seasonality` 的参数传递。如下方法设置每周季节性的先验规模:

```

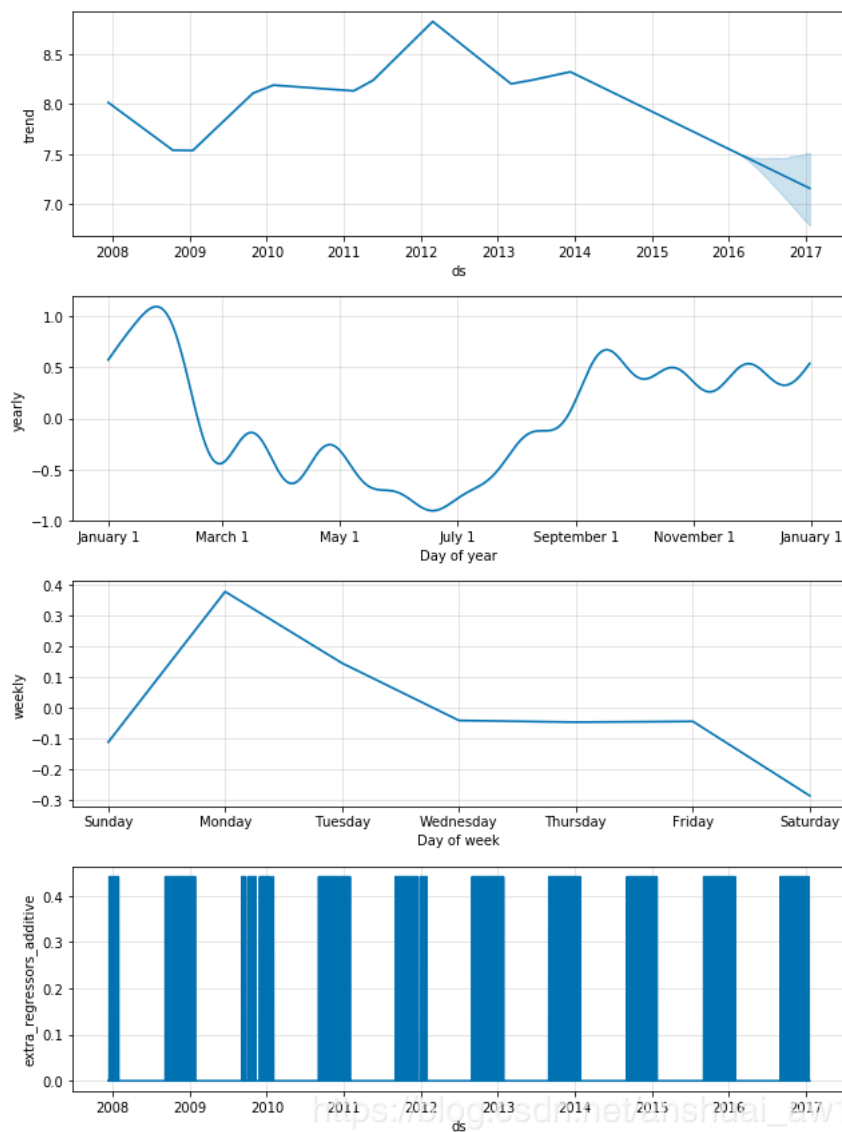
1 m = Prophet()
2 m.add_seasonality(
3     name='weekly', period=7, fourier_order=3, prior_scale=0.1)

```

4.5 附加的回归量

可以使用`add_regressor`方法将附加的回归量添加到模型的线性部分。包含回归值的列需要同时出现在拟合数据格式 (`fit`) 和预测数据格式(`predict`) 们可以在NFL赛季的周日添加附加的效果。在成分图上, 这种效果会出现在 “`extra_regre_`” 图中:

```
1 # 判断是否是NFL赛季的周日
2 def nfl_sunday(ds):
3     date = pd.to_datetime(ds)
4     if date.weekday() == 6 and (date.month > 8 or date.month < 2):
5         return 1
6     else:
7         return 0
8 df['nfl_sunday'] = df['ds'].apply(nfl_sunday)
9
10 m = Prophet()
11 m.add_regressor('nfl_sunday')
12 m.fit(df)
13
14 future['nfl_sunday'] = future['ds'].apply(nfl_sunday)
15
16 forecast = m.predict(future)
17 fig = m.plot_components(forecast)
```



NFL周日也可以使用之前描述的“节假日”的接口, 通过创建一个过去和未来NFL周日的list。`add_regressor`函数为定义附加的线性回归函数接口, 特别是它不要求回归函数是二进制指示器。

`add_regressor`函数具有可选的参数, 用于指定先验规模(默认情况下使用节假日先验规模), 和指定是否标准化回归量。`help(Prophet.add_regressor)` 查看相关参数。

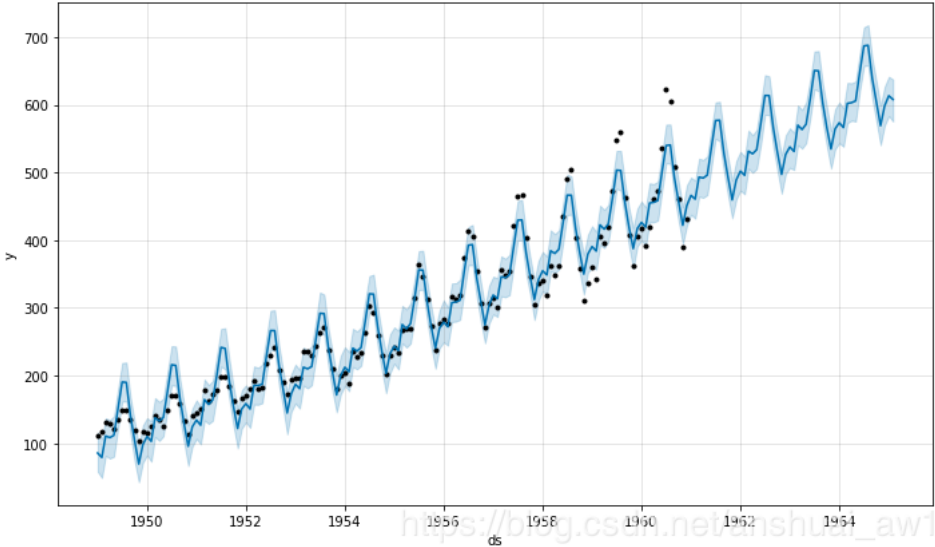
附加的回归量必须要知道历史和未来的日期。因此, 它要么是已知未来值(比如`nfl_sunday`), 要么是其他地方已经单独预测出的结果。如果回归量在整个时间范围内是不变的, 则Prophet会引发一个错误, 因为没有任何东西可以fit它。

附加的回归量被放在模型的线性分量中，所以依赖于附加的回归量时间序列作为底层模型的加法或乘法因子。

五、乘法季节性

默认情况下，Prophet能够满足附加的季节性，这意味着季节性的影响是加到趋势中得到了最后的预报（yhat）。航空旅的时间序列是一个附加作用的例子：

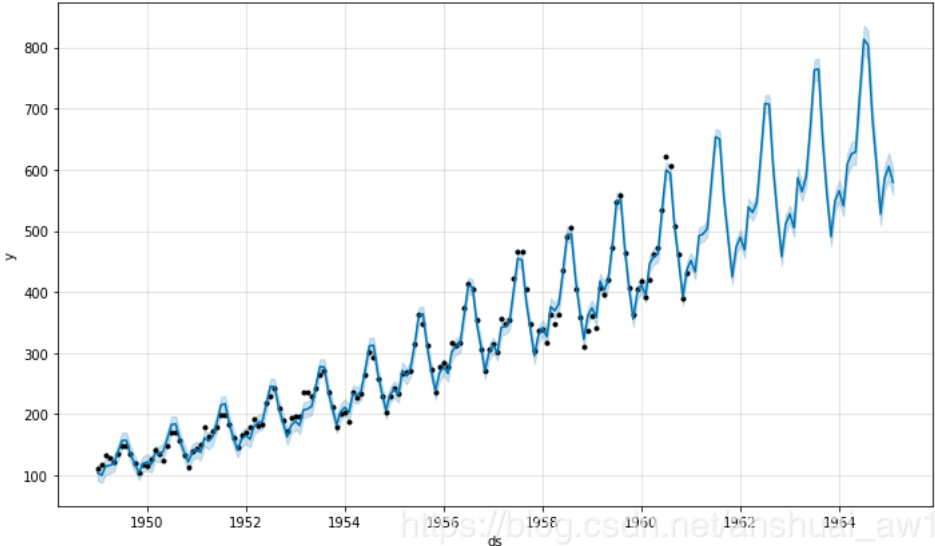
```
1 df = pd.read_csv('../examples/example_air_passengers.csv')
2 m = Prophet()
3 m.fit(df)
4 future = m.make_future_dataframe(50, freq='MS')
5 forecast = m.predict(future)
6 fig = m.plot(forecast)
```



这个时间序列有一个明显的年度周期，但预测中的季节性在时间序列开始时太大，在结束时又太小。在这个时间序列中，季节性并不是Prophet所假定的加性因子，而是随着趋势在增长。这就是乘法季节性（multiplicative seasonality）。

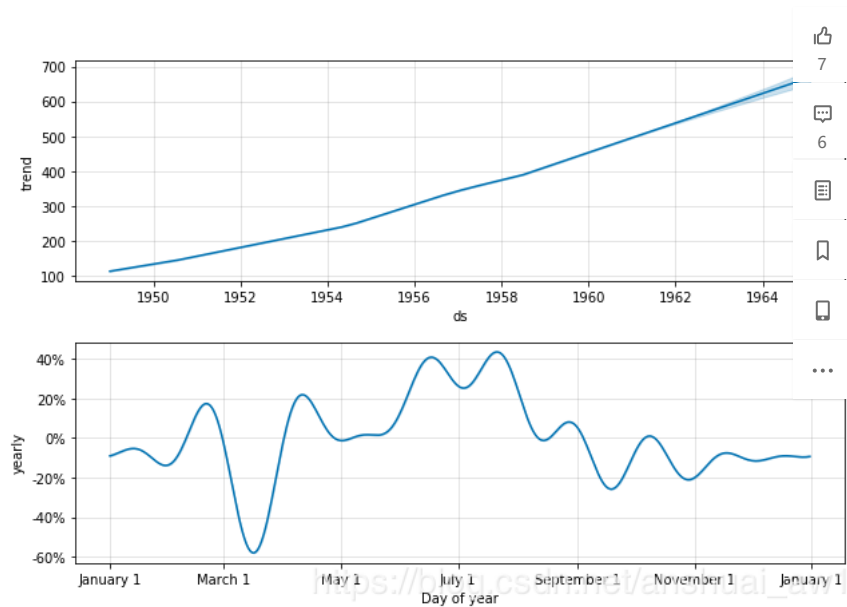
Prophet可以通过设置seasonality_mode='multiplicative'来建模乘法季节性:

```
1 m = Prophet(seasonality_mode='multiplicative')
2 m.fit(df)
3 forecast = m.predict(future)
4 fig = m.plot(forecast)
```



看一下乘法的成分图：

```
fig = m.plot_components(forecast)
```



与第一部分对成分图分析类似，我们这里对乘法模型的成分图进行分析：

①图1是根据trend画出来的，图2是根据yearly画出来的。

②因为是乘法模型，有： $\text{forecast}[\text{'multiplicative_terms'}] = \text{forecast}[\text{'yearly'}]$ ；因此： $\text{forecast}[\text{'yhat'}] = \text{forecast}[\text{'trend'}] * (1 + \text{forecast}[\text{'multiplicative_terms'}])$ 。

使用`seasonality_mode='multiplicative'`，节假日也将被建模为乘法效果。

③因为是乘法模型，`'additive_terms'`，`'additive_terms_lower'`，`'additive_terms_upper'`这3列为0。

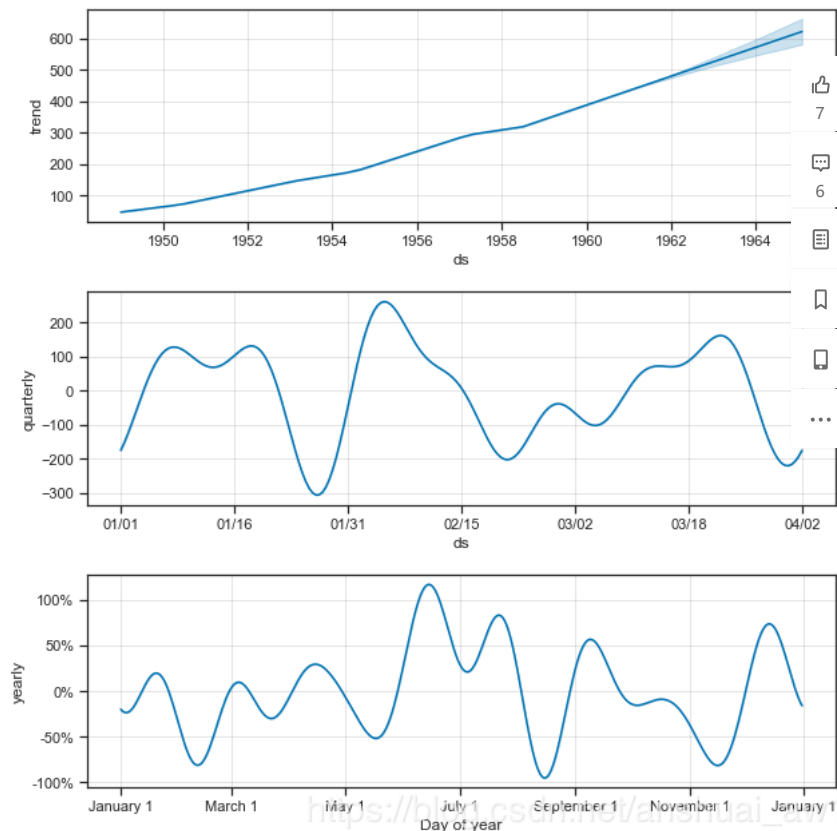
默认情况下，任何添加的季节性或额外的回归量都可以使用`seasonality_mode`设置为加法或者是乘法。但假如在添加季节性或回归量时，可以通过指定`additive`或`mode='active'`作为参数来覆盖之前的设定。

例如，这个模块将内置的季节性设置为乘法，但使用一个附加的季度季节性来覆盖原本的乘法，这时候季度季节性就是加法了。

```
1 | # Python
2 | m = Prophet(seasonality_mode='multiplicative')
3 | m.add_seasonality('quarterly', period=91.25, fourier_order=8, mode='additive')
```

额外的加法的季度季节性将出现在成分图的单独的面板上。





这个时候是时间序列的混合模型： $\text{forecast}[\text{'yhat'}] = \text{forecast}[\text{'trend'}] * (1 + \text{forecast}[\text{'multiplicative_terms'}]) + \text{forecast}[\text{'additive_terms'}]$

六、预测区间

默认情况下，Prophet 的返回结果中会包括预测值 **yhat** 的预测区间。当然，预测区间的估计需建立在一些重要的假设前提下。

在预测时，不确定性主要来源于三个部分：**趋势中的不确定性**、**季节效应估计中的不确定性**和**观测值的噪声影响**。

6.1 趋势的不确定性

预测中，不确定性最大的来源就在于未来趋势改变的不确定性。在之前的时间序列实例中，我们可以发现历史数据具有明显的趋势性。Prophet 能够识别它，但是我们期望得到的趋势改变究竟会如何走向呢？或许这是无解的，因此我们尽可能地做出最合理的推断，假定“未来将会和历史具有相似的趋势”。重要的是，我们假定未来趋势的平均变动频率和幅度和我们观测到的历史值是一样的，从而预测趋势的变化并通过计算，最终得到预测区间。

这种衡量不确定性的方法具有以下性质：变化速率灵活性更大时（通过增大参数 **changepoint_prior_scale** 的值），预测的不确定性也会随之增加。如果将历史数据中更多的变化速率加入了模型，也就代表我们认为未来也会变化得更多，就会使得预测区间成为反映过拟合的标志。

预测区间的宽度（默认下，是 80%）可以通过设置 **interval_width** 参数来控制：

```
1 m = Prophet(interval_width=0.95).fit(df)
2 future = m.make_future_dataframe(periods=365)
3 forecast = m.predict(future)
```

由于预测区间估计时假定未来将会和过去保持一样的变化频率和幅度，而这个假定可能并不正确，所以**预测区间的估计不可能完全准确**。

6.2 季节的不确定性

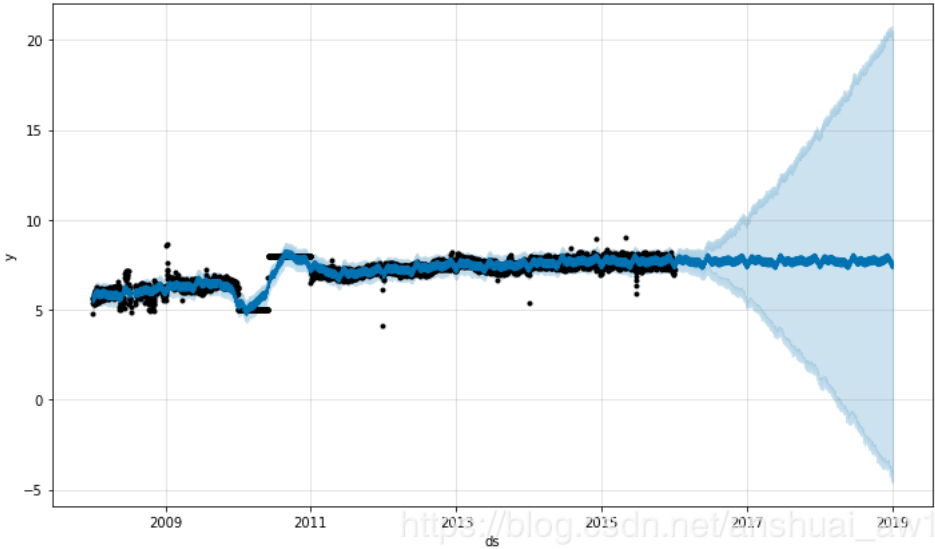
默认情况下，Prophet 只会返回趋势中的不确定性和观测值噪声的影响。你必须使用**贝叶斯取样**的方法来得到季节效应的不确定性，可通过设置 **mcmc.samples** 参数（默认下取 0）来实现。下面使用佩顿·曼宁的数据为例：

```
1 m = Prophet(mcmc_samples=500).fit(df)
2 future = m.make_future_dataframe(periods=365)
3 forecast = m.predict(future)
4 # 报错: UnboundLocalError: local variable 'pool' referenced before assignment
```


七、异常值

异常值主要通过两种方式影响 Prophet 预测结果。下面我们使用之前使用过的 R 语言维基百科主页对数访问量的数据来建模预测，只不过使用存在时整的数据：

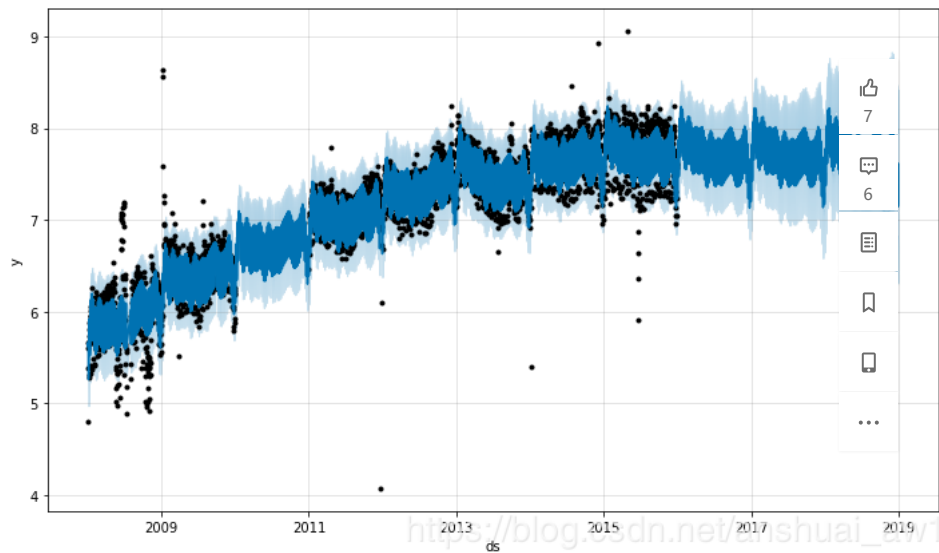
```
1 df = pd.read_csv('examples/example_wp_log_R_outliers1.csv')
2 m = Prophet()
3 m.fit(df)
4 future = m.make_future_dataframe(periods=1096)
5 forecast = m.predict(future)
6 m.plot(forecast);
```



如上输出图所示，趋势预测看似合理，预测区间的估计却过于广泛。Prophet 虽能够处理历史数据中的异常值，但仅仅是将它们与趋势的变化拟合在一起也会有类似的趋势变化。

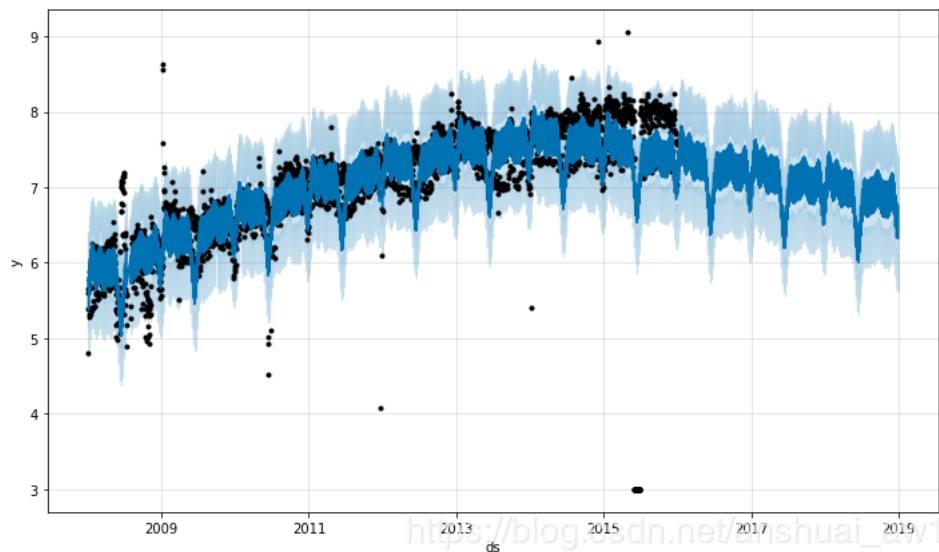
处理异常值最好的方法是移除它们，而 Prophet 是能够处理缺失数据的。如果在历史数据中某行的值为空（NA），但是在待预测日期数据框 future 中日期，那么 Prophet 依旧可以给出该行的预测值。

```
1 # 将2010年一年的数据设为缺失
2 df.loc[(df['ds'] > '2010-01-01') & (df['ds'] < '2011-01-01'), 'y'] = None
3 model = Prophet().fit(df)
4 model.plot(model.predict(future));
```



上述这个实例虽然影响了不确定性的估计，却没有影响到主要的预测值 $\hat{y}$ 。但是，现实往往并非如此，接下来，在上述数据集基础上加入新的异常测：

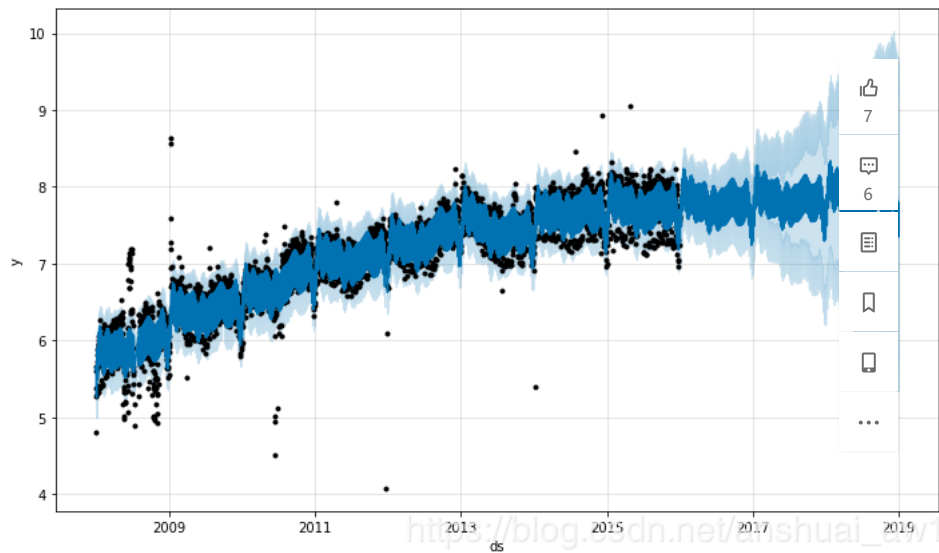
```
1 df = pd.read_csv('examples/example_wp_log_R_outliers2.csv')
2 m = Prophet()
3 m.fit(df)
4 future = m.make_future_dataframe(periods=1096)
5 forecast = m.predict(future)
6 m.plot(forecast);
```



这里 2015年 6 月存在一些异常值破坏了季节效应的估计，因此未来的预测也会永久地受到这个影响。最好的解决方法就是移除这些异常值：

```
1 # 将2015年前半年的数据设为缺失
2 df.loc[(df['ds'] > '2015-06-01') & (df['ds'] < '2015-06-30'), 'y'] = None
3 m = Prophet().fit(df)
4 m.plot(m.predict(future));
```



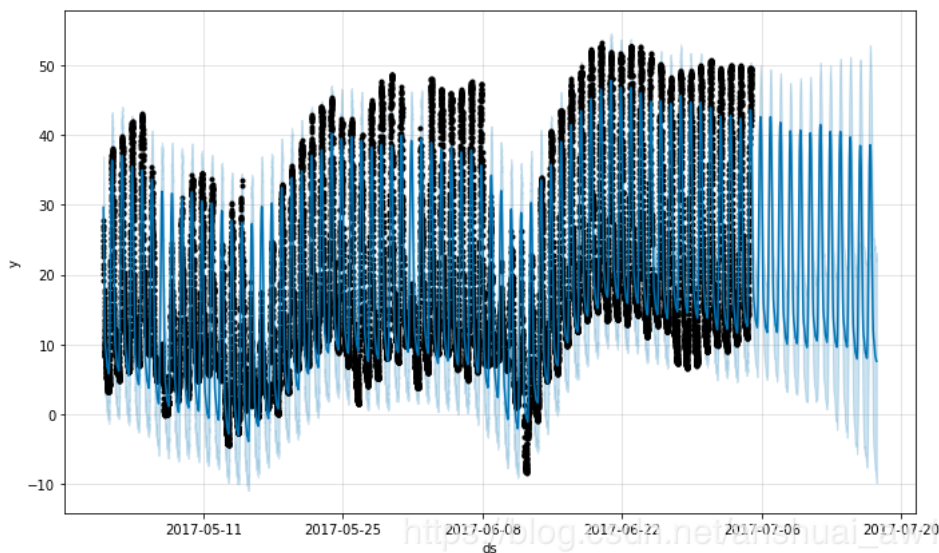


八、非日数据

8.1 子日数据

Prophet可以通过在ds列中传递一个带有时间戳的数据frame来对时间序列进行子日 (Sub-daily) 观测。时间戳的格式应该是YYYY-MM-DD - HH:M用子日数据时，日季节性将自动匹配。在这里，我们用5分钟的分辨率数据集(约塞米蒂的每日温度)对Prophet进行数据匹配:

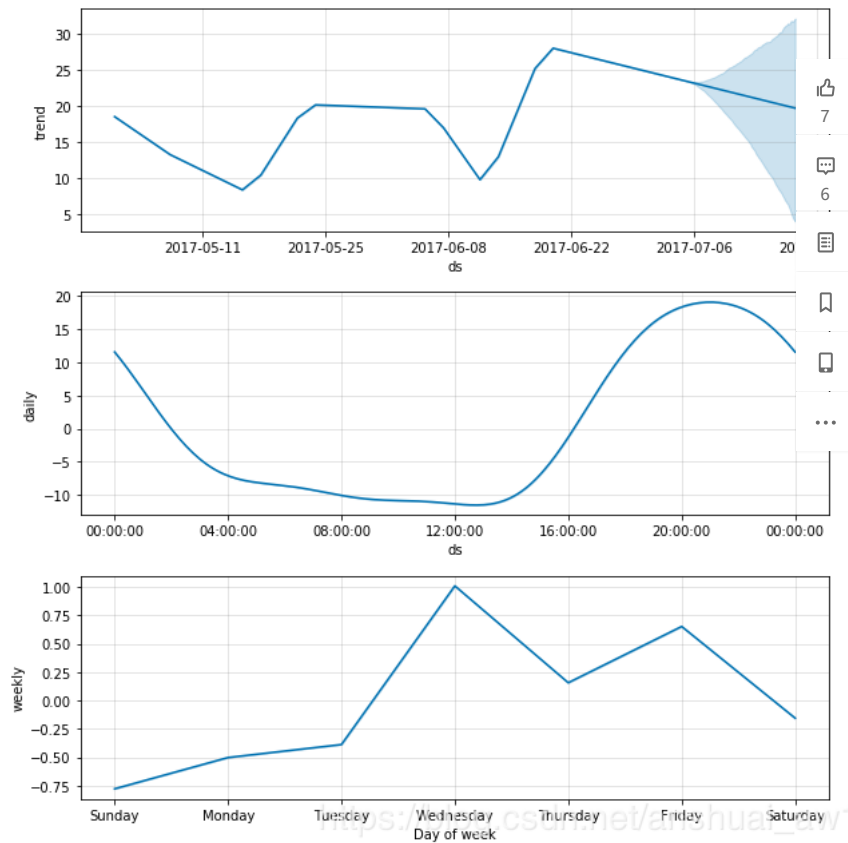
```
1 df = pd.read_csv('examples/example_yosemite_temps.csv')
2 m = Prophet(changepoint_prior_scale=0.01).fit(df)
3 future = m.make_future_dataframe(periods=300, freq='H')
4 fcst = m.predict(future)
5 fig = m.plot(fcst)
```



成分图中的日季节性:

```
fig = m.plot_components(fcst)
```

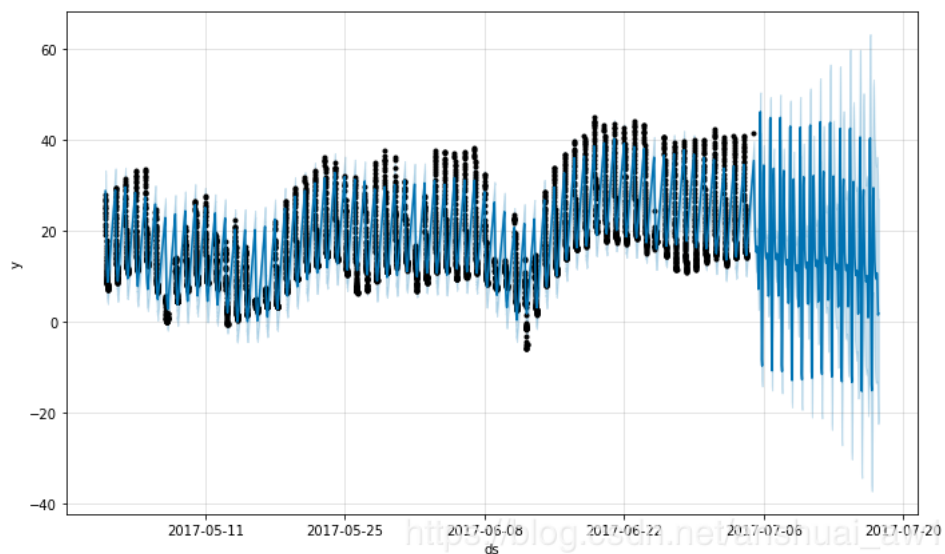




8.2 有规则间隔的数据

假设上面的数据集只有每天早上6点之前的观测值:

```
1 df2 = df.copy()
2 df2['ds'] = pd.to_datetime(df2['ds'])
3 # 只保留每天早上6点之前的数据
4 df2 = df2[df2['ds'].dt.hour < 6]
5 m = Prophet().fit(df2)
6 future = m.make_future_dataframe(periods=300, freq='H')
7 fcst = m.predict(future)
8 fig = m.plot(fcst)
```



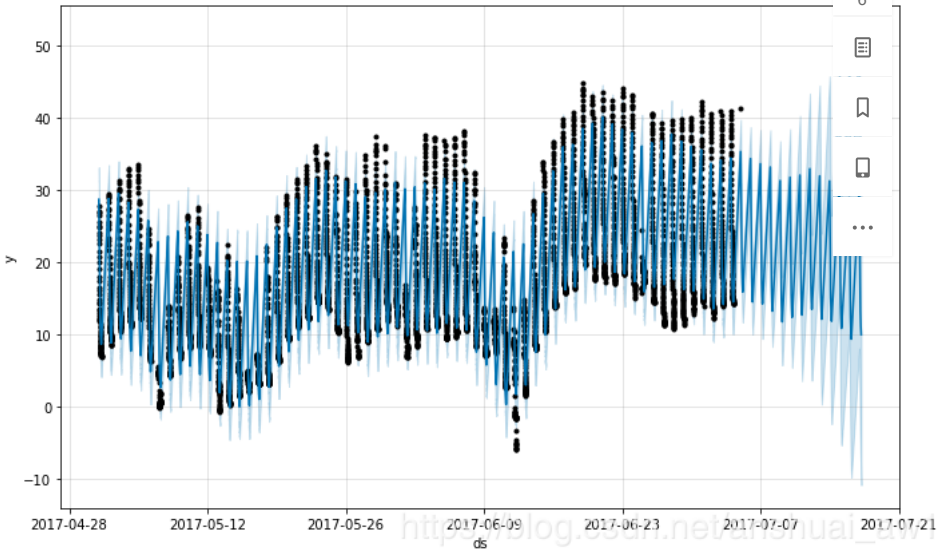
这个预测看起来很差，未来的波动比历史上看到的要大得多。这里的问题是，我们将每天的循环安排在一个时间序列中，这个时间序列中只有(12a到6a)。因此，每天的季节性在一天剩下的时间里是不受约束的，估计也不准确。解决方案是只对有历史数据的时间窗进行预测。这里，这意味着dataframe的时间（从12a到6a）：

```
1 future2 = future.copy()
```

```
2 | future2 = future2[future2['ds'].dt.hour < 6]
3 | fcst = m.predict(future2)
4 | fig = m.plot(fcst)
```



同样的原理也适用于数据中有规则间隔的其他数据集。例如，如果历史只包含工作日，那么应该只对工作日进行预测，因能很好地估计每周的季节

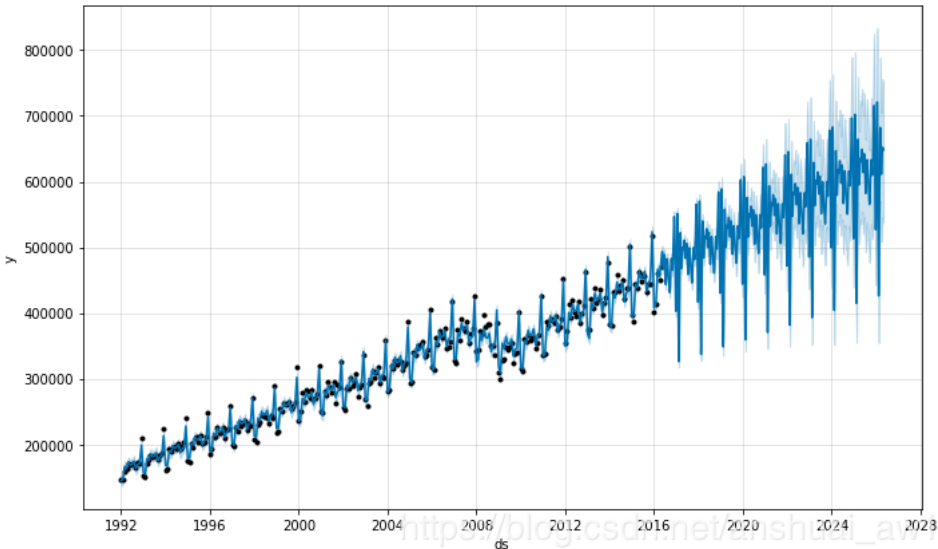


8.3 月数据

可以使用Prophet来匹配每月的数据。然而，Prophet 的基本模型是连续时间的，这意味着如果将模型与每月的数据相匹配，然后要求每天的预测，我的结果。

下面使用美国零售业销售量数据来预测未来 10 年的情况：

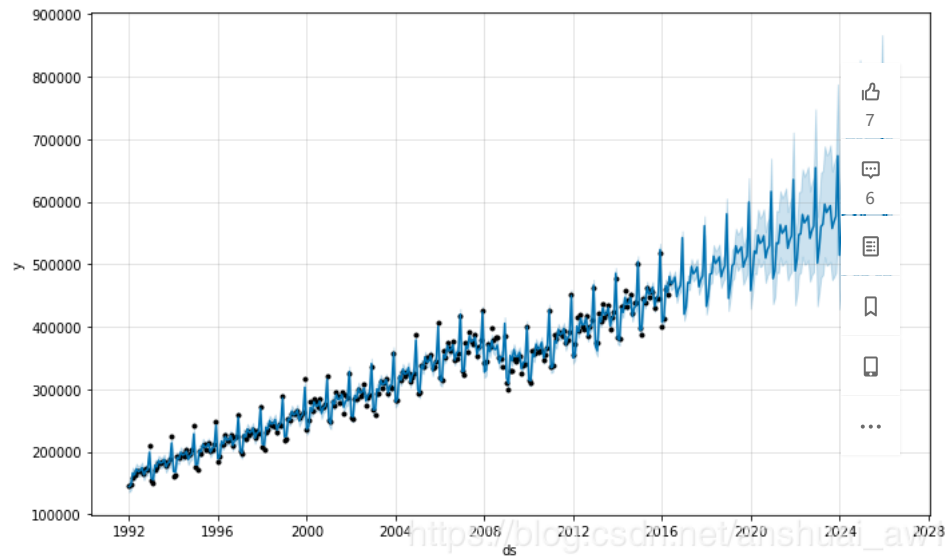
```
1 | df = pd.read_csv('examples/example_retail_sales.csv')
2 | m = Prophet().fit(df)
3 | future = m.make_future_dataframe(periods=3652)
4 | fcst = m.predict(future)
5 | m.plot(fcst);
```



预测结果看起来非常杂乱，原因正是在于这个特殊的数据集使用的是月数据。当我们拟合年度效应时，只有每个月第一天的数据，而且对于其他日子过拟合的。当你使用 Prophet 拟合月度数据时，可以通过在 make_future_dataframe 中传入频率参数只做月度的预测。

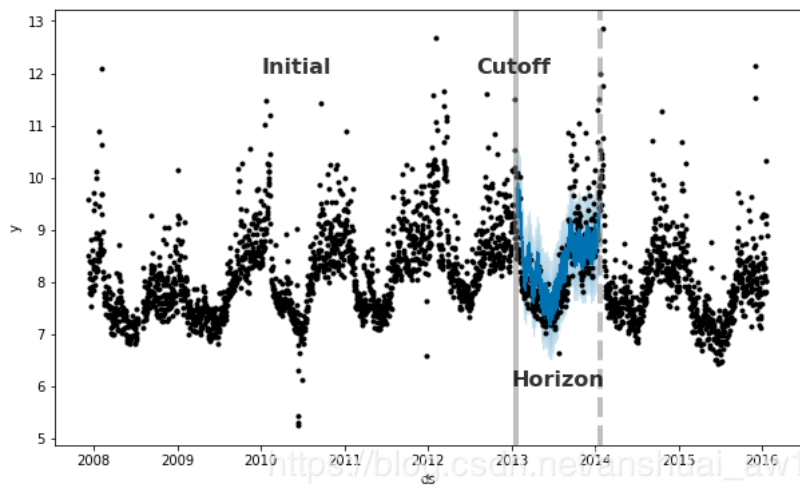
```
1 | future = m.make_future_dataframe(periods=120, freq='M')
2 | fcst = m.predict(future)
3 | m.plot(fcst);
```





九、诊断

Prophet包含时间序列交叉验证功能，以测量使用历史数据的预测误差。这是通过在历史记录中选择截止点来完成的，对于每一个都只使用该截止点之合模型。然后，我们可以将预测值与实际值进行比较。这张图展示了对Peyton Manning数据集的模拟历史预测，该模型适用于5年的初始历史，并在`围内进行了预测。



源论文中进一步描述了模拟的历史预测。

这个交叉验证过程可以使用cross_validation函数自动完成一系列历史截断。我们指定预测水平(horizon)，然后选择初始训练期(initial)的大小和隔(period)。默认情况下，初始训练期设置为horizon的三倍，每半个horizon就有一个截断。

注：这里需要解释下horizon，initial和period的意义：initial代表了一开始的时间是多少，period代表每隔多长时间设置一个cutoff，ho次从cutoff往后预测多少天。

cross_validation的输出是一个dataframe，在每个模拟预测日期和每个截断日期都有真实值y和样本预测值yhat。特别地，对在cutoff和cutoff + 间的每一个观测点都进行了预测。然后，这个dataframe可以用来度量yhat和y的错误。

在这里，我们做交叉验证来评估预测horizon在365天的性能，从第一次截止时730天的训练数据开始，然后每180天进行一次预测。在这个8年的数据中相当于11个总预测。

注：Peyton Manning数据一共大约8年多（2007/12/10 - 2016/01/20）的数据。

根据上面对horizon，initial和period的解释：容易得到为什么是11个总预测。因为最后一个也要预测365天，所有最后一个cutoff在2011-12-19数730天是2010-02-15，则在2010-02-15到2015-01-20共有1800天，对应着10个cutoff，最后1个cutoff在2015-01-20，因此共11个。

```
1 from fbprophet.diagnostics import cross_validation
2 df_cv = cross_validation(m, initial='730 days', period='180 days', horizon = '365 days')
3 df_cv.head()
```


	DS	YHAT	YHAT_LOWER	YHAT_UPPER	Y	CUTOFF
0	2010-02-16	8.957184	8.438130	9.431683	8.242493	2010-02-15
1	2010-02-17	8.723619	8.228941	9.225985	8.008033	2010-02-15
2	2010-02-18	8.607378	8.086717	9.125563	8.045268	2010-02-15
3	2010-02-19	8.529250	8.053584	9.056437	7.928766	2010-02-15
4	2010-02-20	8.271228	7.748368	8.756529	7.745003	2010-02-15

7

6





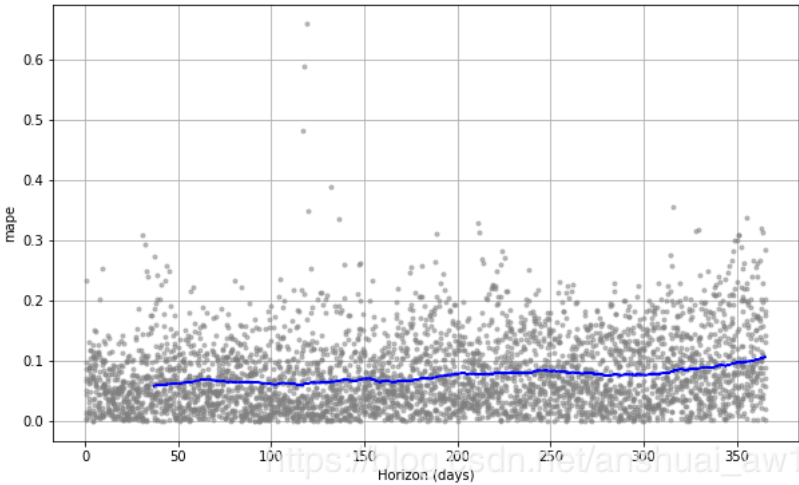
`performance_metrics`作为离截止点(预测的未来距离)的函数，可用于计算关于预测性能的一些有用统计数据(如与`y`相比，`yhat`、`yhat_lower`和`yhat_upper`算得到的统计信息包括均方误差(mean squared error, MSE)、均方根误差(root mean squared error, RMSE)、平均绝对误差(mean absolute error, MAE)以及`yhat_lower`和`yhat_upper`估计的覆盖率。这些都是在`df_cv`中通过`horizon` (`ds - cutoff`)排序后窗口中计算出来的。默认情况下，每个窗口都会包含10%的预测，但是可以通过`rolling_window`参数来更改。

```
1 from fbprophet.diagnostics import performance_metrics
2 df_p = performance_metrics(df_cv)
3 df_p.head()
```

	HORIZON	MSE	RMSE	MAE	MAPE	COVERAGE
3297	37 days	0.481970	0.694241	0.502930	0.058371	0.673367
35	37 days	0.480991	0.693535	0.502007	0.058262	0.675879
2207	37 days	0.480936	0.693496	0.501928	0.058257	0.675879
2934	37 days	0.481455	0.693870	0.502999	0.058393	0.675879
393	37 days	0.483990	0.695694	0.503418	0.058494	0.675879

交叉验证性能指标可以用`plot_cross_validation_metric`可视化，这里显示的是MAPE。点表示`df_cv`中每个预测的绝对误差百分比。蓝线显示的是MAPE取到滚动窗口的圆点。我们可以看到，对于一个月后的预测，误差在5%左右，而对于一年之后的预测，误差会增加到11%左右。

```
1 # Python
2 from fbprophet.plot import plot_cross_validation_metric
3 fig = plot_cross_validation_metric(df_cv, metric='mape')
```



图中滚动窗口的大小可以通过可选参数`rolling_window`更改，该参数指定在每个滚动窗口中使用的预测比例。默认值为0.1，即每个窗口中包含10%的预测。较大的值会导致图中平均曲线更平滑。

初始周期应该足够长，以捕获模型的所有特性，特别是季节性和额外的回归变量:对年的季节性至少保证一年，对周的季节性至少保证一周，等等。

十、与机器学习算法的对比

与先进的机器学习算法如LGBM相比，Prophet作为一个时间序列的工具，优点就是不需要特征工程就可以得到趋势，季节性，节假日因素，但是这个缺点之一，它无法利用更多的信息，如在预测商品的销量时，无法利用商品的信息，门店的信息，促销的信息等。

因此，寻找一种融合的方法是一个迫切的需求。

这项技术，百度/华为等大佬力捧！程序员：我彻底慌了...

没有创造力, 996也打不过AI到底趋势如何？



想对作者说点什么



Python很难：请问一下，这个文章所提到的预测的置信区间是怎样计算的，公式怎样？（2周前 #4楼）



goodgoodstudy99：另外，还烦请问英文原文在哪里下载？前面的链接里的官方文档没有这么详细的内容。（2个月前 #3楼）[查看回复\(1\)](#)



goodgoodstudy99：大神您好！我想请教一个问题，我的数据是月数据，在用prophet进行预测时可通过freq=moth来生成预测集进行预测。但在做交叉验证时，units参数貌似不支持months，这时候该如何做交叉验证呢？请赐教，不胜感激！（2个月前 #2楼）



田志剑伤鸟：您好 做过log处理是为什么吗 要怎么做呢（3个月前 #1楼）[查看问答\(1\)](#)

查看 6 条热评

Facebook时序预测工具Prophet实战分析

阅读数 5780

引言去年Facebook开源了一套时序预测工具叫做Prophet。Prophet是一个预测时间序列数据的模型。它基于一个自... [博文](#) 来自：[OraYang的博客](#)

prophet:时间序列预测模型原理

阅读数 1万+

prophet:时间序列预测原理介绍prophet是Facebook开源一款基于Python和R语言的数据预测工具即“先知”。Fac... [博文](#) 来自：[Jie Qiao的专栏](#)

采用 facebook 的prophet 预测科大讯飞的股票开盘价

阅读数 1854

在运行代码前需要安装fbprophet和tushare,然后拷贝代码可直接运行,个人认为facebook的这个开源不怎么样,回测... [博文](#) 来自：[luoganttc的博客](#)

简述 fbprophet , PyStan库安装。（win10-64位系统）

阅读数 2897

fbprophet依赖于PyStan，所以首先要安装PyStan库，而要安装PyStan，首先要安装C++编译器在自己刚用到的时... [博文](#) 来自：[hhp_lzy的博客](#)

Windows下安装fbprophet

阅读数 1483

环境是win1064位，python3.6.1查看了其他博客的安装方式，安装了pystan。但是在安装fbprophet的时候，碰到... [博文](#) 来自：[weixin_41762358...](#)

win系统下安装Python的fbprophet模块

阅读数 1604

一、按照官方说明fbprophet，windows用户在安装前必需有C++编译器（下载地址：C++），如果电脑里装了vs2... [博文](#) 来自：[Ethan的博客](#)

手把手教你用Prophet快速进行时间序列预测（附Prophet和R代码）

阅读数 4232

作者：ANKITCHOUDHARY；翻译：王雨桐；校对：丁楠雅；本文约3000字，建议阅读12分钟。本文将通过拆解Pr... [博文](#) 来自：[数据分析](#)

用fbprophet预测北京未来一个月的气温

阅读数 694

fbprophet是facebook开源的时序数据预测包，提供了简洁的python和Rapi，可以对时序数据做一些预测，也提供... [博文](#) 来自：[xindoo](#)

prophet:时间序列预测原理

阅读数 1027

prophet:时间序列预测原理介绍prophet是Facebook开源一款基于Python和R语言的数据预测工具即“先知”。Fac... [博文](#) 来自：[hqr20627的博客](#)

时间序列模型Prophet使用详细讲解 - weixin_42053726的..._CSDN博客

这也是一个时间序列处理的模型。下面博客已经讲解了,暂时没有看,以后再学习吧 https://blog.csdn.net/anshuai_aw1/article/details/83412058 涉及到...

...分析之指数平滑法(holt-winters及代码) - anshuai_aw1的博客 - ...

Prophet模型预测商品销售量

阅读数 572

机器学习训练营（qq群号：696721295）——机器学习案例详解的直播互动平台下期直播案例预告：大数据预测商... [博文](#) 来自：[wong2016的博客](#)

prophet模型小结 - fxlou的博客 - CSDN博客

3. The Prophet Forecasting Model prophet模型的构成如下: 其中,是趋势(trend)函数,用来分析时间序列中非周期性的变化。代表周期性的变化,例如一周...

预测学习...

7

6326

6

iw1的博客

(都是血...

列预测 0...

...

prophet:时间序列预测模型原理 - Jie Qiao的专栏 - CSDN博客

时间序列模型Prophet使用详细讲解 10-26 阅读数 8343 之前我们已经讲过了如何...博文 来自: anshuai_aw1的博客 facebook-Prophet-p

时间序列分析之指数平滑法（holt-winters及代码）

在做时序预测时，一个显然的思路是：认为离着预测点越近的点，作用越大。比如我这个月体重100斤，去年某个月1... 博文 来自: anshuai_aw1的博客

Prophet(预言者)facebook时序预测---论文总结以及调参..._CSDN博客

时间序列模型Prophet使用详细讲解 10-26 阅读数 8588 之前我们已经讲过了如何...博文 来自: anshuai_aw1的博客 window下fbprophet

周期性时间序列的预测 - HULK一线技术杂谈 - CSDN博客

时间序列模型Prophet使用详细讲解 10-26 阅读数 6379 之前我们已经讲过了如何...博文 来自: anshuai_aw1的博客 数学建模常用模型06

Linux安装fbprophet的一些错误和解决办法

首先按照官网安装，可以看到需要安装pystan，在对应的页面告诉我们pystan安装需要anaconda，还是先去装anac... 博文 来自: smileglaze的博客

使用Prophet进行时间序列预测

文章概要Prophet是Facebook开源的预测工具，相比ARIMA模型，Prophet真的是非常的简单。只要读入两列数据... 博文 来自: gt9000的博客

时序预测之四 Prophet时序模型 - 谢彦的技术博客 - CSDN博客

时间序列模型Prophet使用详细讲解 10-26 阅读数 6741 之前我们已经讲过了如何...博文 来自: anshuai_aw1的博客 prophet:时间序列预测模型原理 04-1...

数据倾斜及其高效解决方法 - anshuai_aw1的博客 - CSDN博客

博文 来自: anshuai_aw1的博客 mr数据倾斜优化 07-29 阅读数 1149 减小...时间序列模型Prophet使用详细讲解 阅读数 7465 常见的离散型和连续型随机...

R中prophet包说明文档（一）

名称：自动预测过程版本：0.2.1日期：2017-11-08描述：实现了一个时间序列的预测过程，基于能够拟合年度、周... 博文 来自: 莉莉丫丫的博客

类别特征的处理方法大全 - anshuai_aw1的博客 - CSDN博客

本篇博客就介绍了如何利用embedding来构... 博文 来自: anshuai_aw1的博客 ...时间序列模型Prophet使用详细讲解 阅读数 8368 Python:sklearn数据预...

Python: pandas中多级索引的高级操作讲解

Pandas库十分强大，在之前的文章中我已经介绍过了切片操作iloc,loc和ix，本篇文章主要介绍针对多级索引的高级... 博文 来自: anshuai_aw1的博客

初探facebook的prophet时序预测模型

1结合代码对Prophet模型分析1.1模型公式分析&&&& nbsp;y(t)=g(t)+s(t)+h(t)+E... 博文 来自: Thomas_Cai的博客

Prophet文档中文翻译--Seasonality,Holiday effects,and Regressors

%matplotlibinlinefromfbprophetimportProphetimportpandasaspdffrommatplotlibimportpyplotaspltimport... 博文 来自: qq_39315954的博客

【时间序列 - 05】FaceBook / Prophet

Abstract Prophet follows the sklearn model API. We create an instance of the Prophet class and then ... 博文 来自: Houchaoqun_XM...

Facebook大规模时序预测『真』神器—Prophet

本文转载自:http://shujuren.org/article/351.html作者：悟乙己概述：作为经统专业看到预测的packages很眼馋。... 博文 来自: ch的专栏

python-fbprophet总结

时间序列预测一直是预测问题中的难点，人们很难找到一个适用场景丰富的通用模型，这是因为现实中每个预测问... 博文 来自: 阳望的博客

Facebook prophet Python的简单使用

Facebookprophet的使用Prophet意为先知、预言家。ProphetisaforecastingprocedureimplementedinRandPyt... 博文 来自: 星璇的博客

prophet模型小结

1.Introduction在许多自然科学或商业活动中，对于时间序列的预测起着非常重要的作用。但是，做出高精度的预测... 博文 来自: fxlou的博客

python : linux安装fbprophet的脚本

在python中,使用fbprophet进行时间预测,需要安装好多第三方包,比较麻烦的是,好多包有依赖关系,安装的时候错误... 博文 来自: weixin_42367527...

图像处理中的椭圆拟合（opencv）之看的论文的意思的代码

图像处理中的椭圆检测用处还是挺多的，找到这里来的同学大多是想用椭圆检测来解决某些实际问题吧，所以我就不... 博文 来自: eric_e的博客

VIP

🔍

🔔

🛡️

【吐血整理】 windows系统下 fbprophet的安装（ pyhton语言 ）	阅读数 1972
facebook开源了时间序列预测框架prophet，目前支持R语言和python语言。托管在github上：https://github.co... 博文 来自： w	7751221...
windows下安装fbprophet（已解决visual studio 2014报错问题）	阅读数 1066
本人使用windows1064位，python3.5.2，anaconda3.5.3，安装这个机车的函数包，累计耗时快10个小时，总是会... 博文 来自： lixt	ig的博客
Windows下fbprophet的安装	阅读数 316
fbprophet时Facebook开源的时序预测包昨天花了三个小时时间，终于装好了fbprophet，在此记录一下踩过的坑。... 博文 来自： sli	博客
R+python Facebook大规模时序预测『真』神器——Prophet（遍地代码图）	阅读数 1万+
经统专业看到预测的packages都是很眼馋的。除了之前的forecast包，现在这个prophet功能也很强大。本package... 博文 来自： 素	记/Recor...
今天安装 fackbook下面的时间序列模型 prophet	阅读数 467
由于工作需要，所以在linux下安装prophet，本来按照网上的要求安装时很简单的。但是由于公司的linux系统不能... 博文 来自： sjyttkl的专栏	
修改fbprophet源码以支持以小时为粒度的预测	阅读数 4230
在前面那篇文章中介绍了fbprophet的基础运用，但是，那些样本数据都是以天为粒度的。所以，这一篇文章介绍怎... 博文 来自： wujushan的博客	
Android事件分发机制之源码完美解析（上）	阅读数 2万+
学事件分发是为了什么呢？还不是为了解决滑动冲突的。实际上，如果仅仅是为了解决滑动冲突的，大可不必看源码... 博文 来自： qq_36523667的博客	
fbprophet 时间序列预测神器！	阅读数 1625
一、简介： fbprophet是facebook开发的关于时间序列预测的开源工具包，具体的介绍请参考https://facebook.g... 博文 来自： qq_23144435的博客	
Cocos2d-x 2.2.3 使用NDK配置编译环境	阅读数 3万+
Cocos2d-x2.2.3使用NDK配置编译环境2014年6月11日 Cocos2d-x3.0以下的开发环境的配置恐怕折磨了很多... 博文 来自： 巫山老妖	
【小程序】微信小程序开发实践	阅读数 33万+
帐号相关流程注册范围企业政府媒体其他组织换句话说讲就是不让个人开发者注册。:)填写企业信息不能使用... 博文 来自： 小雨同学的技术博客	
【Unity3D Shader编程】之二 雪山飞狐篇：Unity的基本Shader框架写法&颜色、光照与材质	阅读数 9万+
本篇文章中，我们学习了UnityShader的基本写法框架，以及学习了Shader中Properties（属性）的详细写法，光照... 博文 来自： 【浅墨的游戏编程B...	
数据倾斜及其高效解决方法	阅读数 1271
数据倾斜是大数据领域绕不开的拦路虎，当你所需处理的数据量到达了上亿甚至是千亿条的时候，数据倾斜将是横在... 博文 来自： anshuai_aw1的博客	
bsgs算法	阅读数 3万+
bsgs算法bsgs算法，又称大小步算法（某大神称拔山盖世算法）。主要用来解决 $A^x \equiv B \pmod C$ (C是质数)，都是整... 博文 来自： clover_hxy的博客	
【Unity技巧】四元数（Quaternion）和旋转	阅读数 26万+
旋转，应该是三种坐标变换——缩放、旋转和平移，中最复杂的一种了。大家应该都听过，有一种旋转的表示方法叫... 博文 来自： candycat	
Windows下安装Python版本的prophet	阅读数 1310
prophet是Facebook开源的一款时序预测的工具。地址。在Mac和linux下比较好安装，在Windows下安装就比较坑... 博文 来自： anshuai_aw1的博客	
时序预测之四_Prophet时序模型	阅读数 2122
1.说明 Prophet是FaceBook开源的时序框架。非常简单实用，你不需要理解复杂的公式，看图，调参，调用十几行... 博文 来自： 谢彦的技术博客	
5G NR协议栈及功能1 - 总体架构与物理层	阅读数 4万+
备注：本文所有图片均来自3GPP标准，包括3GPPTS38.300V15.0.0(2017-12)和3GPPTS38.202V15.0.0(2017-... 博文 来自： Wei Xingguang	
CLion中使用CMake导入第三方库的方法	阅读数 1万+
由于CLion的工程都是基于CMake来构建的，因此导入第三方库就需要在CMake文件中进行配置。这里把利用CMak... 博文 来自： 大迷毛的LALALAND	
Esp8266学习之旅① 搭建开发环境，开始一个“hellow world”串口打印。	阅读数 7万+
本系列博客学习由非官方人员半颗心脏潜心所力所写，仅仅做个人技术交流分享，不做任何商业用途。如有不对之处... 博文 来自： 徐宏的博客。	
在Tomcat上部署Vue.js项目	阅读数 2万+
接着上次的图书管理系统修改完问题后把SpringBoot成功部署到了服务器上测试了下api接口，能正常访问那么下面... 博文 来自： 小时的枫的博客	

关于计算时间复杂度和空间复杂度

相信学习编程的同学，或多或少都接触到算法的时间复杂度和空间复杂度了，那我来讲讲怎么计算。常用的算法的... 博文 来自： 杨... 阅读量 9万+

编译PROJ4

一、编译PROJ4 PROJ4的最新版本是4.8，官网地址为：http://trac.osgeo.org/proj/。从官网下载PROJ4的源代... 博文 来自： 晴... 阅读量 3939

Java设计模式18——状态模式

一、定义状态(State)模式又称为状态对象模式(PatternofObjectsforState),状态模式是对象的行为模式。状态模式允... 博文 来自： 小... 阅读量 1万+

thymeleaf模板实现html5标签的非严格检查

一、概述最近在springboot项目引入thymeleaf模板时，使用非严格标签时，运行会报错。默认thymeleaf模板对ht... 博文 来自： Luc... 阅读量 6万+

MATLAB计算矩阵间的欧式距离（不用循环！）

MATLAB编程题题目描述：从一个NxM的矩阵C中找出与1xM的矩阵P欧氏距离最小的某一行row，要求不能用循环... 博文 来自： henryzhihua 阅读量 2万+

DirectX修复工具增强版

最后更新：2019-5-26DirectX修复工具最新版：DirectXRepairV3.8增强版NEW!版本号：V3.8.0.11638大小:107M... 博文 来自： VBcom的专栏 阅读量 214万+

机器学习教程 Objective-C培训 交互设计视频教程 颜色模型 设计制作学习

mysql关联查询两次本表 native底部 react extjs glyph 图标 区块链技术讲解 微信开发讲解视频



anshuai_aw1

关注

原创71 粉丝169 喜欢248 评论98

等级： 博客 5 访问：21万+ 积分：2490 排名：2万+ 勋章：  

最新文章

PU learning简介（附python代码）

深入理解L0，L1和L2正则化

分类问题中类别不平衡问题的有效解决方法

Spark性能调优详解

适合工业界的特征选择方法介绍

个人分类

Python20篇

机器学习47篇

开发工具6篇

概率论与数理统计8篇

https://blog.csdn.net/anshuai_aw1/article/details/83412058









26/27

深度学习	15篇
展开	
归档	
2019年4月	5篇
2019年3月	7篇
2019年2月	2篇
2019年1月	7篇
2018年12月	15篇
展开	

热门文章

三大抽样分布：卡方分布，t分布和F分布的简单理解
阅读数 57575

如何简单理解概率分布函数和概率密度函数？
阅读数 22665

Python中的lambda和apply用法
阅读数 9618

时间序列模型Prophet使用详细讲解
阅读数 9449

Python中if __name__ == '__main__', __init__ 和self 的解析
阅读数 6761

最新评论

Python中if __name__...
qq1368334540：非常优秀！

（一）理解word2vec：原理篇
weixin_37807603：分析得很细致呀。

Python中if __name__...
a655400love：这哥们儿厉害，一点就通

时间序列模型Prophet使用详细...
weixin_44593046：请问一下，这个文章所提到的预测的置信区间是怎样计算的，公式怎样？

Xgboost近似分位数算法
Elaine_DWL：请问这里的近似算法和lightgbm中的直方图是一个意思吗？



CSDN学院



CSDN企业招聘

 QQ客服

 kefu@csdn.net

 客服论坛

 400-660-0108

工作时间 8:30-22:00

关于我们 招聘 广告服务 网站地图

 百度提供站内搜索 京ICP备19004658号

©1999-2019 北京创新乐知网络技术有限公司

网络110报警服务 经营性网站备案信息

北京互联网违法和不良信息举报中心

中国互联网举报中心 家长监护 版权申诉

 7

 6









 VIP





